

LAB : 12

Title : Pointers in C++

► Objectives

- To understand computer memory and variables.
- To learn pointer declaration in C++.
- To apply referencing (&) and dereferencing (*) operators.
- To explore pointer arithmetic with arrays.

► Theory

1. Computer Memory

- Memory is divided into bytes, each with a unique address.
- A variable reserves a block of memory depending on its type (e.g., int = 4 bytes on 32-bit systems).
- The compiler maps variable names to memory addresses.

2. Pointers

- A pointer is a variable that stores the address of another variable.
- Declaration syntax:

```
dataType *ptr;
```

- Referencing operator (&) → gives the address of a variable.

- Dereferencing operator (*) → accesses or modifies the value stored at the address.

3. Pointer Arithmetic

- Pointers can be incremented or decremented to move across memory addresses.
- Useful for traversing arrays without using indices

► Lab Tasks

Task 1: Pointer Declaration and Initialization

```
#include <iostream>    //tells the compiler to include input output stream library
```

```
using namespace std;    // tells the compiler to include standard namespace
```

```
int main() {            // main function begins from here
```

```
    int num = 25;        // declare integer variable
```

```
    int *ptr;            // declare pointer to int
```

```
    ptr = &num;          // store address of num in ptr
```

```
    cout << "Value of num: " << num << endl;    // display num directly
```

```
    cout << "Value via ptr: " << *ptr << endl;    // display value using pointer
```

```
    cout << "Address of num: " << ptr << endl;    // display address stored in ptr

    return 0;                                     // end program
}
```

Task 2: Pointer Arithmetic with Arrays

```
#include <iostream>    //tells the compiler to include input output stream library

using namespace std;    // tells the compiler to include standard namespace


int main() {          // main function begins from here

    int numbers[5] = {10, 20, 30, 40, 50};    // array declaration and initialization

    int *ptr;          // declare pointer

    ptr = numbers;     // point to first element of array


    cout << "Array elements using pointer:" << endl;    // output prompt

    for(int i = 0; i < 5; i++) {              // for loop that runs 5 times

        cout << *(ptr + i) << " ";          // access elements using pointer arithmetic

    }


    cout << endl;          // moves compiler to next line

    return 0;             // end program

}
```


► Results

- Learned how pointers store addresses of variables.
- Demonstrated dereferencing to access values through pointers.
- Applied pointer arithmetic to traverse arrays without indices.
- Observed how memory addresses change as the pointer moves across array elements.